Assignment

Compiler Design, Fall - 2025

1. Eliminate left recursion from the following grammar and write the equivalent non-left-recursive grammar suitable for predictive parsing:

E → E+T | E-T | T

T→ T*F | T/F | F

F →(E) | id

2. Eliminate left recursion from the following grammar:

A → AaB | Ab | c

B→ Bd | e

3. Apply the left factoring technique to the following grammar to make it suitable for top-down parsing:

domain → web | website | webapp | diu.bd | edu.bd | diu.edu | edu.ac

4. Apply the left factoring technique to the grammar:

S → abc | abd | aef | g

5. For the input string: "36 july 2024", determine whether the CFG is ambiguous or not.

S → D M Y
D → NN
M → july
Y → NNNN
N → 3 | 6 | 2 | 0 | 4

6. Describe the phases of a compiler for the second statement of the given function.

```
flaot Calc (float a, float b float c, int x){
flaot ans, char ch=c;
ans = a*x+b*x+c /* this is second statement*/
retunr ans;
}
```

7. Draw NFA from the following regular expressions and find the tuples.

i. MN*T | XY+
ii. Pr (P | r)+ rP

8. Consider the following CFG, What will be the output of FIRST() and FOLLOW() functions considering the non-terminals as parameter? Also Construct the LL(1) parse table for the grammar.

E → E+T | E-T | T

T→ T*F | T/F | F

F →(E) | id

9. Consider the following CFG, What will be the output of FIRST() and FOLLOW() functions considering the non-terminals as parameter? Also Construct the LL(1) parse table for the grammar.

E → 5*T | 5+T
T → MS
S → QM | ε
Q → + | *
M → a | b | c

10. Consider the following CFG, What will be the output of FIRST() and FOLLOW() functions considering the non-terminals as parameter? Also Construct the LL(1) parse table for the grammar.

Expense → Tax Salary
Salary → Grocery Tax Salary | / | ε
Grocery → + | - | ε
Tax → Factor Benefit
Benefit → Cost Factor Benefit | / | ε
Cost → *
Factor → (Expense) | num

11. Consider the following grammar to produce LR (0) parser and Canonical Table from the following grammar.

A → AxyB | xC
B → Cbxy | a
C → CbC | y
D → a | m

12. Consider the following grammar, first Augment the above grammar and build all the LR(0) items for the augmented Grammar. Construct the ACTION and GOTO tables using the canonical collection of items.

A → A*BA | xC#y
B → z,BC | CuA
C → BvA | *zD
D → xy,

13. Consider the following expression: **a = (-c * b) + (-c * d)**

i. Draw the Syntax tree for the expression.
ii. Draw the DAG for the expression.
iii. Write the three address code for the expression.
iv. Write the triple for the expression.

14. Consider the following CFG and answer the following questions.

S → PQRT
P → om
R → rc | cr | icr | RQ |€
Q → a | e | i | o | u | €
T → m | n

i. Construct LMD and RMD for the input string “omicron”

ii. Determine if the above-mentioned CFG is ambiguous or not.

15. Consider the following three address code and answer the questions.

1. i = m-1	18. goto (27)	35. goto (24)
2. j = n	19. t9 = a[t8]	36. t19 = a[t18]
3. t1 = 4 * n	20. a[t7] = t9	37. a[t17] = t19
4. goto (2)	21. t10 = 4 * j	38. t20 = 4 * j
5. v= a[t1]	22. a[t10] = x	39. t11 = 4 * n goto (35)
6. i = i+1	23. goto (2)	40. goto (12)
7. t2 = 4 * i	24. t11 = 4 * i	41. v= a[t21]
8. t3 = a [t2]	25. x = a[t11]	42. i = i+1
9. If t3<v goto (7)	26. t12 = 4 * i	43. t22 = 4 * i
10. goto (13)	27. goto (2)	44. t23 = a [t2]
11. t4 = 4 * j	28. t12 = 4 * i	45. If t23<v goto (2)
12. t5 = a[t4]	29. a[t12] = t14 goto (19)	46. goto (24)
13. if t5>v goto (9)	30. t15 = 4 * n	47. t24 = 4 * j
14. if i>=j goto (36)	31. a[t15] = x	48. t25 = a[t24]
15. t6 =4 * i	32. t16 =4 * i	49. if t25>v goto (9)
16. x = a[t6]	33. x = a[t16]	50. if i>=j goto (39)
17. t7 = 4 * i	34. t17 = 4 * i	35. goto (24)

i. List down the general leader selection rules for intermediate code generation.
ii. Which lines of the above code are leader instructions by leader selection rule 2? (Write down the line numbers only).
iii. Which lines of the above code are leader instructions by leader selection rule 3? (Write down the line numbers only).
iv. How many basic blocks are present in the above code?
v. Is there any instruction lines which are selected as leader more than 2 times?

16. Convert the following arithmetic expression into various intermediate representations12:

((rate \times (vat + tax)) - ((cost * tax) + (revenue + (vaf + tax)))

a. Represent the expression using Three Address Code.
b. Represent the expression using Quadruples.
c. Implement a Triples data structure to represent the expression.
d. Construct the Syntax Tree for the expression.
e. Create a Directed Acyclic Graph (DAG) for the expression.

17. Consider the following expression:

x + x * (y-z) + (y-z) * p + q - x + z

a. Write the Three Address Code.
b. Write the representation using Quadruples.
c. Write the representation using Indirect Triples.
d. Draw the DAG for the above expression.

18. Consider the following three address code -

```
int main()
   int a = 10;
   int b = 2;
   int c = a + b;
   int f = a + b;
   int i = f;
   while (i < 20)
      int t2 = a * b;
      i++;

   if (c < 0)  i = e + f;
   return temp2 + i;
```

a. Relate the optimization techniques (e.g., Constant Folding, Common Subexpression Elimination) that can be applied to the above code.

b. Rewrite the code after applying these optimization techniques for maximum performance.

19. Describe the following Code Optimization techniques with proper examples**:**

a. Copy Propagation

b. Strength Reduction

20. Explain how the Constant Propagation optimization technique could be implemented in the code.

21. Apply the Dead Code Elimination technique to the following Three Address Code:

1. a = b + 1
2. b = c * 2
3. d = b - a
4. b = 5
5. output(d)

22. Explain the difference between Local Optimization and Global Optimization.